LAPORAN PRAKTIKUM STRUKTUR DATA

“Penerapan Operasi Stack (LIFO) di Java”

disusun Oleh:

Az Zahrand Solichul Tajussalathin

NIM 2511532001

Dosen Pengampu: Dr. Wahyudi S.T M.T

Asisten Pratikum: Jovantri Immanuel Gulo

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

DAFTAR ISI

DAFTAR ISI i

KATA PENGANTAR ii

BAB I PENDAHULUAN 1

1.1 Latar Belakang 1

1.2 Tujuan Praktikum 1

1.3 Manfaat Praktikum 1

BAB II PEMBAHASAN 3

2.1 Program stackArray 3

2.2 Program stackArrayDriver 4

2.3 Program StackPostfix 4

2.4 Program NilaiMaksimum 5

2.5 Program SiswaStack 6

BAB III KESIMPULAN 9

3.1 Ringkasan Hasil Praktikum 9

DAFTAR PUSTAKA iii

KATA PENGANTAR

Puji syukur kehadirat Allah SWT atas limpahan rahmat dan karunia-Nya sehingga laporan praktikum dengan judul “Penerapan Operasi Stack (LIFO) di Java” ini dapat disusun dan diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pelaksanaan kegiatan praktikum pada mata kuliah Struktur Data.

Ucapan terima kasih disampaikan kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu, serta kepada asisten laboratorium yang telah memberikan bimbingan selama kegiatan praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi penulisan maupun isi. Oleh karena itu, saran dan kritik yang bersifat membangun sangat diharapkan demi perbaikan di masa yang akan datang. Harapannya, laporan ini dapat memberikan manfaat dan menjadi referensi bagi pembelajaran konsep perulangan dalam bahasa pemrograman Java.

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Dalam ilmu komputer dan perancangan algoritma, urutan pemrosesan data tidak selalu dilakukan dari depan ke belakang secar. Terdapat banyak skenario di mana elemen data yang terakhir kali disimpan justru harus menjadi elemen pertama yang diambil atau diproses. Konsep ini dikenal dengan prinsip Last In, First Out (LIFO), yang diimplementasikan melalui struktur data Stack.

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

1. Memahami konsep dasar struktur data abstrak Stack beserta prinsip operasi utamanya yaitu Last In, First Out (LIFO) dalam bahasa pemrograman Java.
2. Mengimplementasikan operasi-operasi fundamental pada Stack, meliputi instruksi push (menambahkan elemen), pop (mengeluarkan elemen teratas), peek (membaca elemen teratas tanpa menghapusnya), serta pengecekan kondisi kosong (isEmpty).
3. Mengaplikasikan struktur data Stack untuk menyelesaikan berbagai studi kasus komputasional praktis.

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini antara lain:

1. Meningkatkan kemampuan dalam mengenali pola masalah komputasi yang membutuhkan pendekatan penyelesaian berbasis Stack.
2. Memberikan pengalaman praktis mengenai bagaimana membangun sebuah struktur data abstrak (ADT) mandiri menggunakan tipe data fundamental (Array) maupun koleksi bawaan Java

3. Menambah pemahaman untuk membedah dan memahami cara kerja compiler atau sistem operasi dalam mengelola alokasi memori.

BAB 2

PEMBAHASAN

2.1 Program stackArray

```
package pekan3_2511532001;

public class stackArray_2511532001 {
	static final int MAX = 1000;
	int top;
	int a[] = new int [MAX];
	boolean isEmpty_2001()
	{
		return (top < 0);
	}
	stackArray_2511532001()
	{
		top = -1;
	}
	boolean push_2001 (int x)
	{
		if (top >= (MAX - 1)) {
			System.out.println("Stack Overflow");
			return false;
		}
		else {
			a[++top] = x;
			System.out.println(x + " dimasukkan dalam stack");
			return true;
		}
	}
	int pop_2001()
	{
		if (top < 0) {
			System.out.println("Stack Underflow");
			return 0;
		}
		else {
			int x = a[top--];
			return x;
		}
	}
	int peek_2001()
	{
		if (top < 0) {
			System.out.println("Stack Underflow");
			return 0;
		}
		else {
			int x = a[top];
```

```
                    return x;
            }
        }
        void print_2001() {
            for(int i = top;i>-1;i--) {
                System.out.print(" " + a[i]);
            }
        }
}
```

Program stackArraymerupakan implementasi dasar dari struktur data Stack dengan menggunakan memori statis berupa Array satu dimensi dengan kapasitas maksimum (MAX) sebesar 1000 elemen. Kelas ini mendefinisikan variabel top yang diinisialisasi dengan nilai -1 sebagai pointer atau penunjuk indeks elemen teratas. Program ini mengimplementasikan mekanisme validasi untuk mencegah eksekusi ilegal, yaitu kondisi Stack Overflow (saat array penuh dan dilakukan operasi push) serta Stack Underflow (saat array kosong dan dilakukan operasi pop atau peek).

2.2 Program stackArrayDriver

```
package pekan3_2511532001;

public class stackArrayDriver_2511532001 {
        public static void main (String[]args) {
            stackArray_2511532001 s = new stackArray_2511532001();
            s.push_2001(10);
            s.push_2001(20);
            s.push_2001(30);
            System.out.println(s.pop_2001() + " dikeluarkan dari stack");
            System.out.println("Elemen teratas adalah: " + s.peek_2001());
            System.out.println("Element pada stack: ");
            s.print_2001();
        }

}
```

Program stackArrayDriver berfungsi sebagai driver atau program utama untuk menguji fungsionalitas dari kelas stackArray. Pada pengujian ini, data dimasukkan secara berurutan bernilai 10, 20, dan 30 menggunakan method push. Sesuai dengan prinsip Last In, First Out (LIFO), ketika method pop dipanggil, elemen yang dikeluarkan dan dikembalikan oleh fungsi adalah 30, yang merupakan elemen terakhir yang disisipkan.

2.3 Program StackPostfix

```java
package pekan3_2511532001;
import java.util.Scanner;
import java.util.Stack;

public class StackPostfix_2511532001 {
	public static int postfixEvaluate_2511532001(String expression) {
		Stack<Integer> s_2001 = new Stack<Integer>();
		Scanner input = new Scanner (expression);
		while (input.hasNext()) {
			if (input.hasNextInt()) {
				s_2001.push(input.nextInt());
			} else {
				String operator = input.next();
				int operand2 = s_2001.pop();
				int operand1 = s_2001.pop();
				if (operator.equals("+")) {
					s_2001.push(operand1 + operand2);
				} else if (operator.equals("-")) {
					s_2001.push(operand1 - operand2);
				} else if (operator.equals("*")) {
					s_2001.push(operand1 * operand2);
				} else {
					s_2001.push(operand1 / operand2);
				}
			}
		}
		input.close();
		return s_2001.pop();
	}
	public static void main (String[]args) {
		System.out.println("hasil postfix: "+
postfixEvaluate_2511532001("5 2 4 * + 7 - "));
	}
}
```

Program StackPostfix mendemonstrasikan implementasi praktis dari struktur data Stack, khususnya dalam mengevaluasi ekspresi Postfix Berbeda dengan program sebelumnya, program ini memanfaatkan library java.util.Stack. Logika kerjanya menggunakan perulangan dengan kelas Scanner untuk membaca setiap token string: jika token berupa angka (operand), data di-push ke dalam tumpukan; jika token berupa operator matematika (+, -, *, /), program akan melakukan pop dua kali untuk mengambil dua elemen teratas, melakukan operasi aritmatika, dan melakukan push kembali hasil perhitungan tersebut ke dalam tumpukan hingga iterasi selesai.

2.4 Program NilaiMaksimum

```java
package pekan3_2511532001;
```

```java
import java.util.Stack;

public class NilaiMaksimum_2511532001 {
	public static int max_2511532001(Stack<Integer> s_2001) {
		Stack<Integer> backup = new Stack<Integer>();
		int maxValue = s_2001.pop();
		backup.push (maxValue);
		while (!s_2001.isEmpty()) {
			int next = s_2001.pop();
			backup.push(next);
			maxValue = Math.max(maxValue, next);
		}
		while (!backup.isEmpty()) {
			s_2001.push(backup.pop());
		}
		return maxValue;
	}
	public static void main (String[]args) {
		Stack<Integer> s_2001 = new Stack<Integer>();
		s_2001.push(70);
		s_2001.push(12);
		s_2001.push(20);
		System.out.println("isi stack "+ s_2001);
		System.out.println("Stack Teratas "+ s_2001.peek());
		System.out.println("Nilai maksimum "+ max_2511532001(s_2001));
	}
}
```

Program NilaiMaksimum memecahkan masalah pencarian nilai terbesar dari sekumpulan data di dalam tumpukan tanpa harus menghapus data aslinya. Algoritma ini berjalan dengan mengekstrak nilai satu per satu menggunakan operasi pop dan membandingkannya menggunakan method Math.max(). Tantangan utama dari Stack adalah datanya akan hilang jika di-pop, sehingga program ini memanfaatkan Stack sekunder bernama backup untuk menyimpan sementara elemen yang dikeluarkan. Setelah proses pencarian selesai, data dari backup dikembalikan (di-push ulang) ke Stack awal sehingga struktur dan urutan datanya tetap utuh.

2.5 Program SiswaStack

```java
package pekan3_2511532001;
import java.util.ArrayList;

class Siswa_2001 {
	String nama_2001;
	int nim_2001;

	public Siswa_2001(String nama_2001, int nim_2001) {
```

```
            this.nama_2001 = nama_2001;
            this.nim_2001 = nim_2001;
        }

        @Override
        public String toString() {
            return "Nim: " + nim_2001 + ", Nama: " + nama_2001;
        }
}

public class SiswaStack_2511532001 {
        private ArrayList<Siswa_2001> stack_2001;

        public SiswaStack_2511532001() {
            stack_2001 = new ArrayList<Siswa_2001>();
        }

        public void push(Siswa_2001 mhs) {
            stack_2001.add(mhs);
        }
        public Siswa_2001 pop_2001() {
            if (!isEmpty_2001()) {
                return stack_2001.remove(stack_2001.size() - 1);
            }
            return null;
        }
        public Siswa_2001 peek_2001() {
            if (!isEmpty_2001()) {
                return stack_2001.get(stack_2001.size() - 1);
            }
            return null;
        }

        public boolean isEmpty_2001() {
            return stack_2001.isEmpty();
        }

        public void tampilkanSiswa_2001() {
            for (int i = stack_2001.size() - 1; i >= 0; i--) {
                System.out.println(stack_2001.get(i));
            }
        }
        public static void main(String[]args) {
            SiswaStack_2511532001 studentStack_2001 = new
SiswaStack_2511532001();

            Siswa_2001 mhs1_2001 = new Siswa_2001("Ali", 1);
            Siswa_2001 mhs2_2001 = new Siswa_2001("Boby", 2);
            Siswa_2001 mhs3_2001 = new Siswa_2001("Charles", 3);

            studentStack_2001.push(mhs1_2001);
            studentStack_2001.push(mhs2_2001);
            studentStack_2001.push(mhs3_2001);
```

```
            System.out.println("Siswa di dalam stack: ");
            studentStack_2001.tampilkanSiswa_2001();

            System.out.println("siswa teratas " +
studentStack_2001.peek_2001());
            System.out.println("mengeluarkan siswa teratas dari stack :" +
studentStack_2001.pop_2001());
            System.out.println("daftar siswa setelah di pop:");
            studentStack_2001.tampilkanSiswa_2001();
        }
}
```

Program SiswaStack mengombinasikan kelebihan alokasi memori dinamis dari ArrayList dengan aturan pembatasan akses data ala Stack. Kelas ini tidak menyimpan tipe data primitif, melainkan objek kompleks berupa Siswa (User-Defined Object). Enkapsulasi digunakan dengan mendeklarasikan ArrayList stack sebagai atribut private. Operasi manipulasi difasilitasi melalui method push, pop, dan peek yang dirancang untuk selalu menargetkan elemen pada indeks terakhir (size() - 1).

BAB 3

KESIMPULAN

3.1 Ringkasan Hasil Praktikum

Berdasarkan praktikum dan implementasi program yang telah dilakukan, dapat disimpulkan bahwa struktur data Stack merupakan solusi komputasional yang krusial untuk mengelola data dengan prinsip Last In, First Out (LIFO). Praktikum ini membuktikan bahwa Stack dapat diimplementasikan menggunakan memori statis (Array manual) maupun dinamis (menggunakan ArrayList atau kelas bawaan java.util.Stack). Pada implementasi berbasis Array statis, pengelolaan pointer atau indeks top menjadi kunci utama, di mana mekanisme validasi mutlak diperlukan untuk mencegah anomali memori seperti Stack Overflow saat penambahan data melampaui kapasitas, dan Stack Underflow saat penghapusan pada tumpukan kosong. Operasi fundamental seperti push(), pop(), dan peek() terbukti berguna dalam memecahkan studi kasus algoritmik, seperti evaluasi ekspresi matematika Postfix dan pencarian nilai maksimum menggunakan stack backup. Selain itu, penerapan prinsip enkapsulasi pada Stack untuk mengelola tipe data objek (user-defined object seperti kelas Siswa) merupakan hal yang krusial untuk membatasi akses dan menjaga integritas data di dalam memori.

DAFTAR PUSTAKA

[1] W3Schools, “Stacks”. [Daring].

Tersedia pada: https://www.w3schools.com/dsa/dsa_data_stacks.php.

[Diakses: 10 April 2026].